

Spécifications fonctionnelles — User Stories

Projet : SmartTicket — Gestionnaire de tickets intelligent avec assistant virtuel

Stack : Next.js / FastAPI / PostgreSQL+pgvector / Mistral API

Référentiel d'accessibilité : WCAG 2.1 niveau AA

US-01 : Poser une question en langage naturel à l'assistant

En tant que Client

Je veux saisir une question en langage naturel dans le widget de chat

Afin de obtenir une aide immédiate sans connaître la structure du système de tickets

Contexte

Le client est authentifié sur le portail de support (session JWT valide). Il rencontre un problème avec un produit ou service et souhaite une réponse rapide. Une session de chat est active (statut **open**) et aucun opérateur humain n'est encore impliqué.

Scénarios d'utilisation

Scénario nominal :

Le client ouvre l'interface de chat → saisit sa question dans le champ de texte → appuie sur Entrée ou clique le bouton Envoyer → la question est persistée en base (**type_envoyeur** = 'user') avec horodatage → l'indicateur de chargement s'affiche → le pipeline RAG est déclenché.

Scénario alternatif :

Le client pose une question de suivi dans une session déjà ouverte → la question est contextualisée avec l'historique existant de la session → la réponse tient compte des échanges précédents.

Scénario d'échec :

Le client tente d'envoyer un message vide ou dépassant 2 000 caractères → un message d'erreur est affiché immédiatement ("Le message ne peut pas être vide" / "Message trop long") → aucun appel API n'est effectué.

Critères d'acceptation

- **[Fonctionnel]** Le champ de saisie accepte du texte en langage naturel sans contrainte de syntaxe ; aucune commande spéciale n'est requise.
- **[Fonctionnel]** Le message est persisté en base avec **type_envoyeur** = 'user' et horodaté avant le déclenchement du pipeline RAG.
- **[Performance]** L'indication de chargement s'affiche en moins de 500 ms après l'envoi du message.
- **[Sécurité]** Le contenu du message est sanitisé côté serveur (suppression de balises HTML, protection XSS) avant stockage et traitement.
- **[Accessibilité WCAG 2.1 AA — critère 2.1.1 Clavier]** L'envoi du message est réalisable intégralement au clavier (touche Entrée sur le champ, ou Tab + Espace/Entrée sur le bouton Envoyer), sans nécessiter de souris ni de dispositif de pointage.
- **[Accessibilité WCAG 2.1 AA — critère 4.1.2 Nom, rôle, valeur]** Le champ de saisie expose **role="textbox"**, un **aria-label="Votre message"** et **aria-required="true"** ; le bouton Envoyer

expose `role="button"` et un `aria-label` explicite.

US-02 : Recevoir une réponse pertinente issue de la base documentaire

En tant que Client

Je veux recevoir une réponse générée à partir de la base de connaissances via le pipeline RAG

Afin de résoudre mon problème sans intervention humaine

Contexte

Suite à l'envoi d'une question (US-01), le pipeline RAG est déclenché côté serveur. Le client voit un indicateur de chargement animé pendant la génération. La base documentaire (pgvector) contient des articles pertinents pour la question posée et les embeddings sont calculés par Mistral Embed (dimension 1 024).

Scénarios d'utilisation

Scénario nominal :

Le serveur encode la question via Mistral Embed → effectue une recherche par similarité cosinus dans pgvector (HNSW, top K=4) → construit le prompt RAG avec les chunks récupérés → appelle Mistral LLM en mode streaming → la réponse s'affiche token par token dans le widget de chat.

Scénario alternatif :

La réponse du LLM mentionne une source documentaire → l'interface affiche un lien cliquable vers l'article source pour que le client puisse approfondir.

Scénario d'échec :

Le modèle Mistral est indisponible (timeout > 30 s ou erreur 503) → un message d'erreur explicite est affiché ("L'assistant est temporairement indisponible") avec un bouton "Parler à un opérateur".

Critères d'acceptation

- **[Fonctionnel]** La réponse est générée via RAG en utilisant les 4 chunks documentaires les plus proches selon la similarité cosinus sur l'index HNSW de pgvector.
 - **[Fonctionnel]** La réponse s'affiche en mode streaming (Server-Sent Events, token par token) sans attendre la génération complète.
 - **[Performance]** Le premier token de la réponse est affiché en moins de 3 secondes au 95e percentile, mesurés depuis la réception de la requête côté serveur.
 - **[Sécurité]** Le prompt RAG envoyé au LLM est construit exclusivement côté serveur ; le client ne peut pas modifier ni injecter de contexte documentaire via l'API.
 - **[Accessibilité WCAG 2.1 AA — critère 4.1.3 Messages d'état]** L'indicateur de chargement ("L'assistant génère une réponse...") est annoncé aux lecteurs d'écran via `aria-live="polite"` ; la fin de génération déclenche également une annonce ("Réponse disponible").
 - **[Accessibilité WCAG 2.1 AA — critère 1.4.3 Contraste minimum]** Le texte de la réponse du bot respecte un ratio de contraste $\geq 4,5:1$ sur fond clair et $\geq 4,5:1$ sur fond sombre (mode dark), vérifiable via l'outil WebAIM Contrast Checker.
-

US-03 : Être escaladé vers un opérateur humain quand le bot ne peut pas répondre

En tant que Client

Je veux demander le transfert de ma conversation vers un opérateur humain

Afin d' obtenir une aide personnalisée pour un problème complexe ou sensible que le bot ne résout pas

Contexte

Le client a reçu des réponses insuffisantes ou rencontre un problème nécessitant une intervention humaine (technique, sensible, contractuel). La session est en statut **open**. Un bouton "Parler à un opérateur" est accessible dans l'interface de chat à tout moment.

Scénarios d'utilisation

Scénario nominal :

Le client clique "Parler à un opérateur" → une modale s'ouvre demandant le motif (technique / complexe / sensible / autre) → le client confirme → la session passe au statut **transferred** avec le **transfer_reason** renseigné → une notification est envoyée aux opérateurs SAV → le client voit "Vous êtes en file d'attente, un opérateur va vous répondre".

Scénario alternatif :

Le bot détecte une impossibilité de répondre (aucun chunk pertinent trouvé, score de similarité < seuil) → une suggestion proactive de transfert s'affiche ("Je n'ai pas trouvé de réponse satisfaisante. Souhaitez-vous parler à un opérateur ?") → le client confirme.

Scénario d'échec :

Aucun opérateur disponible (hors horaires d'ouverture) → le système affiche les horaires d'assistance et propose de laisser un message qui sera traité à l'ouverture.

Critères d'acceptation

- **[Fonctionnel]** La session passe au statut **transferred** avec le champ **transfer_reason** renseigné parmi les valeurs autorisées : **technique, complexe, sensible, autre**.
- **[Fonctionnel]** Une notification est envoyée à tous les utilisateurs avec le rôle **sav** dès que le transfert est effectué.
- **[Performance]** La confirmation du transfert est affichée en moins de 2 secondes après la soumission de la demande.
- **[Sécurité]** Seul l'utilisateur propriétaire de la session (vérification **id_utilisateur** extrait du JWT) peut initier le transfert de sa propre session.
- **[Accessibilité WCAG 2.1 AA — critère 4.1.3 Messages d'état]** La confirmation de transfert ("Vous êtes en attente d'un opérateur") est annoncée par **aria-live="assertive"** pour alerter immédiatement les utilisateurs de lecteurs d'écran, sans attendre une interaction.
- **[Accessibilité WCAG 2.1 AA — critère 3.3.2 Étiquettes ou instructions]** La modale de sélection du motif de transfert comporte un **<label>** explicite pour chaque option radio, avec une instruction textuelle décrivant les cas d'usage de chaque motif.

US-04 : Suivre l'état d'un ticket

En tant que Client

Je veux consulter le statut de mes sessions de support en cours et passées

Afin de savoir où en est ma demande sans avoir à contacter le support

Contexte

Le client est authentifié. Il a créé une ou plusieurs sessions de support dans le passé. Il accède à la liste de ses tickets depuis son espace personnel, accessible via le menu principal.

Scénarios d'utilisation

Scénario nominal :

Le client navigue vers "Mes tickets" → une liste paginée affiche ses sessions avec : titre, statut (open / transferred / resolved / closed), date de création, date de dernière modification → le client clique sur un ticket → il accède au détail de la conversation complète.

Scénario alternatif :

Le client filtre ses tickets par statut → seules les sessions correspondant au filtre sélectionné sont affichées → il peut réinitialiser le filtre.

Scénario d'échec :

Le client n'a aucun ticket → le composant affiche "Aucune conversation en cours" avec un bouton primaire "Démarrer une nouvelle demande".

Critères d'acceptation

- **[Fonctionnel]** Le statut de chaque ticket est affiché avec son libellé textuel (open → "En cours", transferred → "Transféré", resolved → "Résolu", closed → "Fermé") et une couleur codée distincte.
- **[Fonctionnel]** La liste est triée par date de dernière modification décroissante avec pagination de 20 éléments par page.
- **[Performance]** La liste des tickets s'affiche en moins de 1 seconde au 95e percentile pour un client ayant jusqu'à 500 sessions.
- **[Sécurité]** Un client ne peut voir que ses propres tickets : le filtre `id_utilisateur` est appliqué côté serveur depuis le JWT, jamais depuis un paramètre client.
- **[Accessibilité WCAG 2.1 AA — critère 1.4.1 Utilisation de la couleur]** Le statut du ticket n'est pas identifiable uniquement par la couleur : chaque badge de statut comporte un libellé textuel lisible ("Transféré", "Résolu...") en complément de la couleur.
- **[Accessibilité WCAG 2.1 AA — critère 4.1.2 Nom, rôle, valeur]** La liste de tickets est structurée en éléments `<ul>` / `<li>` sémantiques ; chaque badge de statut expose un `aria-label` complet (ex : `aria-label="Statut : Transféré"`).

US-05 : Reprendre une conversation laissée par le bot

En tant qu' Opérateur (SAV)

Je veux consulter et reprendre une session transférée depuis le bot

Afin de fournir une aide personnalisée au client en ayant le contexte complet de la conversation

Contexte

L'opérateur est connecté avec le rôle `sav`. Son tableau de bord liste les sessions avec statut `transferred`, triées par ancienneté (les plus anciennes en premier). L'opérateur peut cliquer sur une session pour consulter

l'historique avant d'intervenir.

Scénarios d'utilisation

Scénario nominal :

L'opérateur accède à son dashboard → voit la liste des sessions transférées avec motif + horodatage → clique sur une session → lit l'historique complet (messages client + réponses bot) → saisit sa réponse dans le champ de message → envoie (`type_envoyeur = 'sav'`) → la conversation continue.

Scénario alternatif :

L'opérateur résout le problème du client → clique "Marquer comme résolu" → la session passe au statut `resolved` → le client est notifié → le bot peut potentiellement reprendre la session.

Scénario d'échec :

La session a été prise en charge par un autre opérateur entre-temps → un indicateur "En cours de traitement" est affiché avec le nom de l'opérateur qui a pris en charge → le second opérateur ne peut pas envoyer de message simultanément.

Critères d'acceptation

- **[Fonctionnel]** L'opérateur voit l'intégralité des messages de la session (client + bot) avant d'envoyer son premier message, sans limite de pagination sur l'historique de la session courante.
- **[Fonctionnel]** Les messages envoyés par l'opérateur sont sauvegardés avec `type_envoyeur = 'sav'` et horodatés en UTC.
- **[Performance]** L'historique complet d'une session (jusqu'à 200 messages) se charge en moins de 2 secondes.
- **[Sécurité]** Seuls les utilisateurs avec le rôle `sav` ou `admin` peuvent accéder aux sessions transférées d'autres utilisateurs ; vérification stricte du rôle côté serveur (403 pour tout autre rôle).
- **[Accessibilité WCAG 2.1 AA — critère 1.3.1 Information et relations]** Les messages sont structurés sémantiquement : chaque bulle identifie son auteur (client / bot / opérateur) via un élément `<header>` ou `<span>` avec `aria-label`, lisible par les lecteurs d'écran sans se fier uniquement à la position visuelle.
- **[Accessibilité WCAG 2.1 AA — critère 2.4.6 En-têtes et étiquettes]** Le panneau d'historique des messages comporte un titre de section descriptif (`<h2>`) ; un lien d'ancrage "Aller au dernier message" permet d'atteindre directement le bas de la conversation.

US-06 : Évaluer la qualité d'une réponse du bot pour alimenter l'apprentissage

En tant qu' Opérateur (SAV)

Je veux noter positivement ou négativement une réponse du bot via le système de feedback

Afin d' alimenter les métriques de qualité de l'IA et identifier les réponses à améliorer

Contexte

L'opérateur consulte une session transférée et analyse les réponses que le bot a fournies au client. Chaque message avec `type_envoyeur = 'ai'` affiche des boutons de feedback (pouce en haut / pouce en bas).

Scénarios d'utilisation

Scénario nominal :

L'opérateur lit une réponse bot → clique le bouton "Réponse utile" (feedback = 1) ou "Réponse non utile" (feedback = -1) → le score est persisté en base → le bouton actif est mis en évidence visuellement (couleur + `aria-pressed="true"`).

Scénario alternatif :

L'opérateur change d'avis après avoir cliqué → il reclique sur le bouton opposé → le feedback est mis à jour (écrasement de la valeur précédente).

Scénario d'échec :

L'opérateur tente de soumettre un feedback sur un message client ou opérateur (`type_envoyeur ≠ 'ai'`) → les boutons de feedback ne sont pas affichés pour ces messages ; l'API retourne 400 si la requête est forcée.

Critères d'acceptation

- **[Fonctionnel]** Le feedback (valeur `1` ou `-1`) est persisté sur le champ `feedback` du message correspondant dans la table `chat_messages`.
- **[Fonctionnel]** Les feedbacks sont agrégés dans les métriques du dashboard : score de satisfaction = nombre de feedbacks positifs / nombre total de feedbacks avec valeur.
- **[Performance]** L'enregistrement du feedback est confirmé visuellement en moins de 500 ms après le clic.
- **[Sécurité]** Seuls les utilisateurs avec le rôle `sav` ou `admin` peuvent soumettre des feedbacks ; l'endpoint vérifie le rôle côté serveur (403 sinon).
- **[Accessibilité WCAG 2.1 AA — critère 4.1.2 Nom, rôle, valeur]** Les boutons de feedback sont des éléments `<button>` natifs avec `aria-label="Réponse utile"` / `aria-label="Réponse non utile"` et `aria-pressed="true|false"` indiquant l'état actif.
- **[Accessibilité WCAG 2.1 AA — critère 2.4.7 Visibilité du focus]** Le focus clavier sur les boutons de feedback est visible avec un contour d'au moins 2 px ; le ratio de contraste du contour de focus par rapport à l'arrière-plan adjacent est $\geq 3:1$.

US-07 : Gérer la base documentaire — ajouter / modifier / supprimer un article

En tant qu' Administrateur

Je veux ajouter, modifier ou supprimer des sources de la base de connaissances

Afin de maintenir la base documentaire à jour et améliorer la pertinence des réponses du bot

Contexte

L'administrateur est connecté avec le rôle `admin`. Il accède à l'interface de gestion de la base documentaire depuis le panneau d'administration. Les sources peuvent être des URLs, des sitemaps ou des fichiers uploadés (PDF, DOCX, TXT).

Scénarios d'utilisation

Scénario nominal :

L'admin clique "Ajouter une source" → choisit le type (URL / sitemap / fichier) → saisit l'URL ou upload le fichier → soumet → un job d'ingestion est lancé en arrière-plan → la progression est affichée en temps réel → à la fin : "Ingestion réussie : N chunks créés et indexés".

Scénario alternatif :

L'admin saisit l'URL d'un sitemap → le système extrait automatiquement toutes les URLs listées → chaque page est ingérée individuellement → un rapport final liste le nombre de pages traitées et le nombre de chunks créés.

Scénario d'échec :

L'URL est inaccessible (404) ou le fichier `robots.txt` interdit le scraping → le job échoue avec un message d'erreur explicite ("Ce site interdit l'indexation automatique" ou "URL introuvable") → aucun chunk n'est créé ni indexé.

Critères d'acceptation

- **[Fonctionnel]** L'ingestion d'un fichier PDF / DOCX / TXT produit des chunks de ≈ 500 caractères avec chevauchement de 50 caractères, chacun associé à un embedding `vector(1024)` stocké dans pgvector.
- **[Fonctionnel]** La suppression d'une source supprime tous les chunks associés en cascade (`ON DELETE CASCADE` sur la clé étrangère).
- **[Performance]** L'ingestion d'un document de 50 pages (≈ 25 000 mots) se termine en moins de 120 secondes, incluant le calcul des embeddings Mistral.
- **[Sécurité]** Seul le rôle `admin` peut effectuer des opérations d'ingestion ou de suppression ; l'endpoint retourne 403 pour tout autre rôle.
- **[Accessibilité WCAG 2.1 AA — critère 3.3.1 Identification des erreurs]** En cas d'erreur d'ingestion, le message identifie précisément la cause (URL invalide, format non supporté, robots.txt restrictif) et est associé au champ concerné via `aria-describedby`.
- **[Accessibilité WCAG 2.1 AA — critère 1.4.10 Redistribution]** Le formulaire d'ingestion est utilisable sans défilement horizontal à une largeur de 320 px CSS, sans perte d'information ni de fonctionnalité (critère applicable aux interfaces de saisie et de progression).

US-08 : Consulter le tableau de bord des métriques

En tant qu' Administrateur

Je veux consulter les métriques de performance de l'assistant IA sur le dashboard Analytics

Afin de piloter la qualité du service et identifier les axes d'amélioration

Contexte

L'administrateur est connecté avec le rôle `admin`. Il accède à la page Analytics depuis le menu principal. Les métriques disponibles incluent : taux de résolution IA, score de satisfaction, analyse des transferts par motif, alertes de performance. Les graphiques sont rendus par Recharts.

Scénarios d'utilisation**Scénario nominal :**

L'admin navigue vers "Analytics" → les KPIs principaux sont affichés (taux de résolution IA, score de

satisfaction, nb de sessions, nb de transferts) → des graphiques en courbe et en barres montrent les tendances sur 30 jours → l'admin peut filtrer par période (7 j / 30 j / 90 j).

Scénario alternatif :

L'admin identifie un pic de transferts → il clique sur le graphique des transferts → la liste des sessions transférées pour la période filtrée s'affiche avec le motif de transfert détaillé.

Scénario d'échec :

La base est vide ou la plateforme vient d'être installée → le dashboard affiche les valeurs à 0 avec le message "Données insuffisantes — au moins 10 conversations sont nécessaires pour calculer des métriques fiables".

Critères d'acceptation

- **[Fonctionnel]** Les métriques calculées sont : `taux_resolution_ia = sessions closed / sessions totales`, `score_satisfaction = feedbacks positifs / total feedbacks`, `nb_transferts` ventilé par motif (`technique`, `complexe`, `sensible`, `autre`).
- **[Fonctionnel]** Les données sont chargées depuis `GET /api/analytics` à chaque ouverture de la page ; un bouton "Rafraîchir" permet un rechargement manuel.
- **[Performance]** Le chargement complet du dashboard (données + rendu des graphiques Recharts) se fait en moins de 2 secondes pour un jeu de données de 10 000 conversations.
- **[Sécurité]** L'endpoint `GET /api/analytics` retourne 403 Forbidden pour tout rôle différent de `admin`; aucune donnée agrégée n'est accessible sans authentification.
- **[Accessibilité WCAG 2.1 AA — critère 1.1.1 Contenu non textuel]** Chaque graphique Recharts dispose d'une alternative textuelle : un titre descriptif visible, les valeurs clés exposées sous forme de tableau `<table>` masqué visuellement mais accessible aux lecteurs d'écran, et un `aria-label` sur le conteneur SVG.
- **[Accessibilité WCAG 2.1 AA — critère 1.4.1 Utilisation de la couleur]** Les graphiques n'utilisent pas uniquement la couleur pour distinguer les séries de données : chaque série est également différenciée par une forme de point ou un motif de ligne (plein, tirets, pointillés).